



Tecnológico de Monterrey

Estudiantes:

Axel Javier Rosas Rodríguez | A01738607

Elian Cantalapiedra Sabugal | A01738462

Edwin Emmanuel Salazar Meza | A01738380

Profesor:

Rigoberto Cerino Jiménez

Nombre de la Institución:

Instituto Tecnológico y de Estudios Superiores de
Tecnológico de Monterrey

Materia:

Implementación de Robótica Inteligente

Fecha:

20 de Abril de 2026

Título del trabajo:

Reto semanal 3. Manchester Robotics

Resumen

Se presenta el diseño e implementación de un sistema de control de movimiento para un robot móvil diferencial (Puzzlebot). A través de la técnica de estabilización de punto (Point Stabilisation), el objetivo es llevar al robot a una posición deseada en un plano bidimensional. El proceso integra un esquema de control de lazo cerrado que estima la posición actual del robot mediante odometría (obtenidas de la velocidad de las ruedas) para determinar el error de posición respecto a la meta. A partir de este error, un controlador ajusta dinámicamente las velocidades lineal y angular para guiar al vehículo hasta el objetivo.

Objetivos

Objetivo General:

- Implementar un sistema de control de lazo cerrado para lograr la estabilización de punto en un robot móvil de tracción diferencial, guiándolo de manera autónoma hacia una posición objetivo (x_g) mediante la retroalimentación de su estado actual obtenido por odometría.

Objetivos específicos:

- **Estimar velocidades del robot:** Crear un nodo en ROS que se suscriba a los datos de los encoders de las ruedas para calcular de forma continua las velocidades lineal y angular del robot.
- **Implementar localización:** Desarrollar un sistema de navegación por estima que utilice las velocidades calculadas y el modelo cinemático diferencial para determinar iterativamente la pose del vehículo (x_r, y_r, θ_r) en el plano bidimensional.
- **Calcular errores de estado:** Desarrolló la lógica geométrica necesaria para procesar la diferencia entre la posición estimada del robot y la meta predefinida, obteniendo el error de distancia (e_d) y el error de orientación (e_θ) en cada instante de tiempo.
- **Diseñar la ley de control:** Aplicar un controlador (Proporcional o PID) que asigne ganancias (K_v y K_ω) a los errores calculados, generando comandos de velocidad dinámica (V y ω) que estabilice al robot en la coordenada deseada.
- **Validar la precisión y robustez:** Definir un umbral de tolerancia práctico (ejemplo: 10 cm) para determinar la culminación exitosa de la trayectoria, mitigando el impacto del ruido acumulativo inherente a los sensores odométricos.

Introducción

El control de movimiento en sistemas autónomos busca determinar las entradas necesarias para que un robot alcance un objetivo predefinido en un tiempo finito. En vehículos con restricciones no holonómicas (como un robot de tracción diferencial, que no puede desplazarse lateralmente), la tarea de estabilización en un punto específico (x_g) representa

un desafío de control complejo, ya que no puede lograrse mediante leyes de control de retroalimentación de estado invariantes en el tiempo. Para resolver esto, se requiere la integración de técnicas de localización, cálculo geométrico del error y lazos de control externos.

Metodología

Odometría y Localización

La odometría es el método mediante el cual se estima el cambio de posición del robot basándose en los datos de sus propios sensores. En este caso, los encoders miden la velocidad angular de las ruedas individuales (ω_R y ω_L), lo que permite calcular la velocidad lineal (V) y la velocidad de giro (ω) del centro de masa del robot. Utilizando el modelo cinemático diferencial y el método de integración de Euler, estas velocidades se proyectan sobre los ejes globales para actualizar de forma discreta las coordenadas x, y y la orientación ω en cada instante de tiempo Δt .

Control en Lazo Cerrado para un Robot Móvil

Para controlar el movimiento, se adapta el diagrama clásico de retroalimentación. El "Set Point" se define como el vector de coordenadas objetivo $X_g = (x_g, y_g)^T$. La odometría actúa como el bloque de sensores que provee la salida medida (la pose actual del robot X). Al restar esta posición estimada de la meta deseada, se genera el vector de error (e), el cual es procesado por un controlador externo. Este controlador dicta los comandos de velocidad que sirven de referencia para los controladores de bajo nivel (lazo interno) integrados en los motores de la plataforma.

Controlador PID, Cálculo de Error y Robustez

El error cartesiano se debe traducir a un sistema polar relativo a la perspectiva del vehículo para aplicar el control. Esto se logra calculando el error de distancia como $e_d = \sqrt{e_x^2 + e_y^2}$ y el error de orientación usando $e_\theta = \text{atan2}(e_y, e_x) - \theta_r$. Dado que la plataforma diferencial es inherentemente estable en su dinámica de conducción, una ley de control puramente Proporcional suele ser suficiente como punto de partida: $V = K_v e_d$ y $\omega = K_\omega e_\theta$.

Sin embargo, para dotar al sistema de mayor robustez, se pueden incorporar elementos integrales y derivativos. Esta robustez es necesaria para contrarrestar el ruido aditivo de los sensores y la fricción del entorno; por ejemplo, la naturaleza de la odometría provoca que el error se acumule con el tiempo. Por esta razón, el diseño del controlador no busca que la distancia sea matemáticamente cero, sino que define un umbral de tolerancia (ej. 10 cm) para determinar empíricamente que el vehículo ha llegado con éxito a la meta.

Solución del problema

Para resolver el reto se implementó un sistema de navegación punto a punto para un robot móvil diferencial utilizando ROS 2, basado en una arquitectura jerárquica que separa la planeación de trayectorias del control de bajo nivel.

La solución se divide en dos nodos principales:

PathGenerator

Se encarga de definir los puntos objetivos, calcular el error de posición y generar referencias de movimiento.

ControlLazoCerrado

Implementa controladores PI para regular las velocidades lineal y angular del robot a partir de la retroalimentación de la odometría.

El supervisor recibe la posición del robot desde el tópico /odom y calcula el error de distancia y orientación respecto al punto objetivo

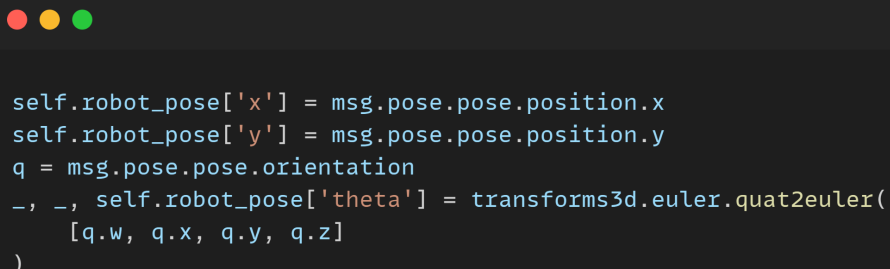
El pathgenerator define una lista de puntos objetivo y recibe la posición actual del robot mediante el tópico /odom:



```
self.targets = [  
    {'x': 2.0, 'y': 0.0},  
    {'x': 2.0, 'y': 2.0},  
    {'x': 0.0, 'y': 2.0},  
    {'x': 0.0, 'y': 0.0}  
]
```

Imagen 1. Definición de objetivos

A partir de la odometría, se calcula la posición y orientación del robot:



```
self.robot_pose['x'] = msg.pose.pose.position.x  
self.robot_pose['y'] = msg.pose.pose.position.y  
q = msg.pose.pose.orientation  
_, _, self.robot_pose['theta'] = transforms3d.euler.quat2euler(  
    [q.w, q.x, q.y, q.z]  
)
```

Imagen 2. Lectura de la odometría

Posteriormente, se calcula el error de posición y orientación hacia el punto objetivo:

```

dx = target['x'] - self.robot_pose['x']
dy = target['y'] - self.robot_pose['y']
dist = np.sqrt(dx**2 + dy**2)

target_angle = np.arctan2(dy, dx)
error_theta = np.arctan2(
    np.sin(target_angle - self.robot_pose['theta']),
    np.cos(target_angle - self.robot_pose['theta'])
)

```

Imagen 3. Cálculo del error

El nodo ControlLazoCerrado recibe las referencias del supervisor y la retroalimentación de velocidad desde la odometría:

```

self.v_real = msg.twist.twist.linear.x
self.w_real = msg.twist.twist.angular.z

if msg.move_type == "turn":
    self.v_ref = 0.0
    self.w_ref = msg.velocity

elif msg.move_type == "advance":
    self.v_ref = msg.velocity
    self.w_ref = 0.0

```

Imagen 4. Velocidad a seguir

El control se realiza mediante controladores PI independientes para velocidad lineal y angular.

Control PI lineal

```

e = v_ref - self.v_real
self.e_v_int += e * self.dt
self.e_v_int = np.clip(self.e_v_int, -0.5, 0.5)

u = self.Kp_v * e + self.Ki_v * self.e_v_int
return float(np.clip(u, -0.4, 0.4))

```

Control PI angular

```

e = w_ref - self.w_real
self.e_w_int += e * self.dt
self.e_w_int = np.clip(self.e_w_int, -0.5, 0.5)

u = self.Kp_w * e + self.Ki_w * self.e_w_int
return float(np.clip(u, -4.0, 4.0))

```

Imagen 5. Control Lineal y angular PI + Anti-Windup

Resultados

Durante la ejecución del sistema, se observó que el robot logra seguir la trayectoria punto a punto definida; sin embargo, se presenta un error angular residual aproximado de 2 a 3 grados en cada giro realizado por el robot. Este error no se corrige completamente y se acumula conforme el robot ejecuta múltiples giros consecutivos, lo cual provoca desviaciones graduales en la orientación y, en menor medida, en la posición final.



Imagen 6. Resultado del control

Este comportamiento se debe principalmente a:

- Limitaciones inherentes de la odometría basada en ruedas, como el deslizamiento.
- Saturación de la velocidad angular durante los giros.
- Uso de una tolerancia angular fija para finalizar la fase de giro.
- Ausencia de sensores externos de corrección de orientación.

A pesar de este error acumulativo, el controlador PI permite que el robot alcance cada punto objetivo dentro de la tolerancia establecida, manteniendo un movimiento estable y sin oscilaciones significativas. El envío del comando de parada final se realiza correctamente

Conclusiones

Se implementó exitosamente un sistema de navegación punto a punto para un robot móvil diferencial utilizando control en lazo cerrado y odometría en ROS 2. El sistema permite al robot alcanzar los puntos objetivos de forma estable, aunque se identificó un error angular sistemático de aproximadamente 2–3 grados por giro, el cual se acumula a lo largo de la trayectoria.

Este error acumulativo es consistente con las limitaciones conocidas de la odometría en robots móviles diferenciales y no invalida el funcionamiento del sistema, sino que evidencia la necesidad de técnicas adicionales de corrección para aplicaciones que requieran mayor precisión. El controlador PI implementado logró compensar parcialmente estos errores, garantizando estabilidad y suavidad en el movimiento.

Como posibles mejoras, se propone la implementación de control simultáneo de velocidad lineal y angular o la corrección dinámica del error angular durante la fase de avance. Estas mejoras permitirían reducir el error acumulativo y aumentar la precisión del sistema.

Referencias

Manchester Robotics Ltd. TE3002B Intelligent Robotics Implementation – Week 3 Challenge.